

Architettura di un'app come «Ricettario»

Guida per rifarne una con la stessa struttura tecnologica

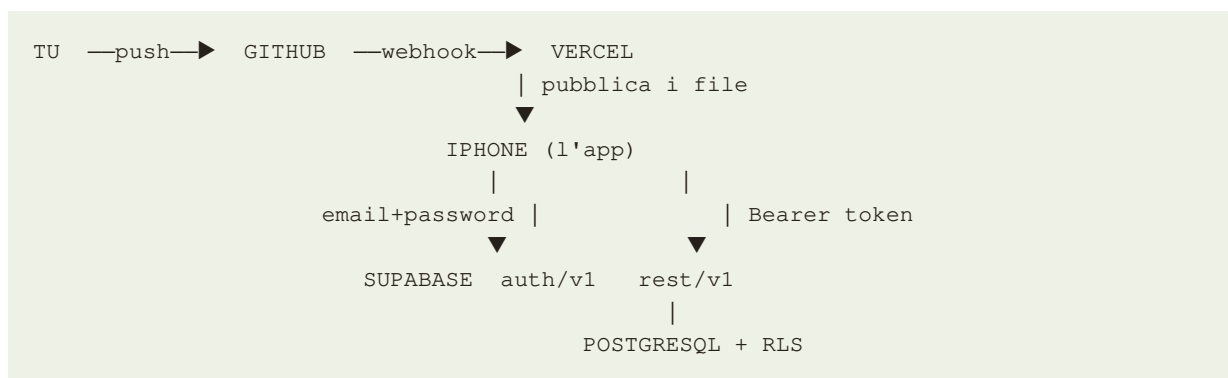
GITHUB · VERCEL · SUPABASE — 08/09/2026

1. L'idea in una riga

Un **sito statico** — solo HTML, CSS e JavaScript, senza alcun passaggio di compilazione — ospitato su **Vercel**, il cui codice vive su **GitHub**, e che parla **direttamente** a un database PostgreSQL su **Supabase** via HTTP.

Il punto sorprendente è che **non esiste un backend scritto da noi**: nessun server, nessuna API da mantenere, nessun contenitore da aggiornare. Il browser dell'utente è il client, e il database espone da sé la propria interfaccia HTTP.

2. Lo schema



Vercel serve solo file. Ogni dato passa dal telefono a Supabase, senza intermediari.

3. Chi fa cosa

GitHub — il sorgente e il telecomando

Conserva il codice e lo storico. Ma soprattutto è il **grilletto del deploy**: Vercel osserva il repository e ripubblica a ogni commit. Non si carica mai niente «sul server» a mano.

Vercel — l'hosting

Distribuisce i file statici su CDN con HTTPS e un dominio già pronto. Con questa architettura la configurazione è **vuota**: preset «Other», nessun comando di build, nessuna cartella di output. Vercel si limita a servire i file così come sono.

Supabase — il database e tutto il resto

Fornisce tre cose che di solito richiederebbero un backend:

- **PostgreSQL** gestito, con un editor SQL nel pannello;

- **Auth**: registrazione e accesso via email e password, che emette token JWT;
- **API REST generata dallo schema**: ogni tabella diventa automaticamente un endpoint HTTP sotto `/rest/v1/`. Questo strato si chiama PostgREST.

4. Il pezzo da capire bene: la sicurezza

Se il database è raggiungibile direttamente dal telefono, cosa impedisce a chiunque di leggerlo? Non il nascondere la chiave: la chiave sta nel JavaScript, in chiaro, e **chiunque può leggerla**.

La difesa è un'altra: le **Row Level Security policy**, regole scritte in SQL dentro il database che dicono chi può vedere e modificare quali righe. Senza una policy che autorizzi, una tabella con RLS attiva non restituisce niente.

Il dettaglio che inganna: una lettura non autorizzata **non** risponde con un errore. Risponde **200 OK** con una lista vuota, `[]`. Quindi «funziona senza errori» non vuol dire «è protetto», e «non vedo dati» non vuol dire «è rotto». Il modo giusto di verificare è chiamare l'endpoint senza token e controllare che il corpo della risposta sia davvero vuoto.

Le due chiavi, da non confondere

- **anon** (o *publishable*): identifica il progetto, sta nel client, è pubblica per definizione. Da sola non dà accesso a niente: passa attraverso le RLS.
- **service_role**: **scavalca** le RLS. Non va mai nel codice del client, né in un repository, nemmeno privato. Si usa solo da un server o dal pannello.

Un esempio di policy

Per un archivio condiviso, in cui ogni persona autenticata vede e modifica tutto:

```
alter table ricette enable row level security;

create policy "condiviso tra utenti autenticati" on ricette
  for all to authenticated using (true) with check (true);
```

Cambiando quella condizione si cambia il modello: `using (user_id = auth.uid())` rende ogni riga privata del suo autore. È la stessa app, con una riga di SQL diversa.

5. Autenticazione: come gira il token

1. L'utente inserisce email e password; il client chiama `/auth/v1/token?`
`grant_type=password`.
2. Supabase restituisce un **access token** (un JWT che scade, tipicamente in un'ora), un **refresh token** di durata lunga e la scadenza del primo.
3. Ogni chiamata ai dati porta l'intestazione `Authorization: Bearer <access token>`.
Postgres legge il token, ne ricava l'identità e applica le RLS.
4. Quando l'access token è vicino alla scadenza, si usa il refresh token per ottenerne uno nuovo.

Lezione imparata sul campo: conviene rinnovare il token **prima** di usarlo, controllandone la scadenza, e non **dopo** che la richiesta è già stata rifiutata. Con il rinnovo reattivo ogni salvataggio a token scaduto costa tre viaggi di rete — tentativo fallito, rinnovo, nuovo tentativo — invece di uno, e su rete mobile la differenza si sente tutta. Vale anche la regola complementare: **ritentare solo se l'errore è 401**. Su un errore di rete, ritentare è solo tempo perso prima di dire che non è andata.

La sessione (i due token, l'identificativo utente, la scadenza) si conserva in `localStorage`, così riaprendo l'app non si rifà il login.

6. La forma del codice

Con questa architettura i file sono pochi e hanno confini netti:

- **index.html** — tutto il markup e tutto il CSS. Un file solo: senza build non ci sono fogli di stile da assemblare, e avere lo stile accanto alla struttura riduce gli scarti.
- **app.js** — la logica: stato, disegno delle schermate, gestori degli eventi.
- **supabase.js** — l'unico punto che conosce la rete: indirizzo del progetto, chiave pubblica, e una funzione per operazione (accedi, elenca, inserisci, modifica, elimina). Tenerlo separato significa che il resto dell'app non sa nulla di HTTP.
- **manifest.json** e le icone — servono a far installare l'app sulla schermata Home.
- **schema.sql** e le **migrazioni numerate** — la struttura del database, versionata nel repository accanto al codice che la usa.

Non serve alcun framework. Non è una posizione ideologica: senza build non c'è nulla da aggiornare, nulla che si rompa da sé fra sei mesi, e il file che scrivi è identico al file che il telefono esegue — quindi quello che vedi nel browser è quello che c'è.

7. Il database in pratica

Liste dentro le righe

Per elenchi che appartengono a una sola riga — i passi di un procedimento, gli strumenti selezionati — conviene una colonna **jsonb** invece di una tabella figlia. Si evitano join e la scrittura resta una sola richiesta. La regola pratica: tabella separata solo se quei dati vanno cercati o ordinati da soli.

Migrazioni idempotenti

Ogni modifica è un file numerato che si può rieseguire senza danni:

```
alter table ricette add column if not exists note text not null default '';
```

Lo stesso vale per le importazioni: inserire solo se non esiste già una riga equivalente, così rilanciare lo script non crea doppioni.

Il client va scritto tollerante

Il codice e il database si aggiornano in momenti diversi: il telefono può avere la versione nuova

prima che la migrazione sia stata eseguita, o viceversa. Conviene quindi che il client sopporti campi mancanti e valori che non conosce, invece di rompersi.

8. L'app installata su iPhone

Un sito statico diventa un'icona sulla Home senza pubblicare su alcuno store: bastano un `manifest.json`, un'icona e Safari. Ma iOS ha alcune regole che costa tempo scoprire da soli.

iOS legge il manifest e i meta tag una volta sola, quando aggiungi l'icona alla Home. Non li rilegge mai più, nemmeno ricaricando. Se cambi icona, nome o colori devi **rimuovere e riaggiungere** l'app. Il CSS e il JavaScript, invece, si aggiornano normalmente. Sapere questa differenza evita di inseguire modifiche che sembrano non fare effetto.

Andare davvero a schermo pieno

Tre accorgimenti, ognuno dei quali corrisponde a una banda indesiderata:

- **Nessun theme_color**, né nel manifest né in un meta tag: è quello a disegnare la striscia opaca dietro l'orologio, e in standalone resta del colore letto all'installazione. Insieme a `apple-mobile-web-app-status-bar-style: black-translucent` e `viewport-fit=cover` la pagina passa sotto la barra di stato.
- **Lo sfondo va sull'elemento radice**, `html`, non solo su `body`: se `body` è in `position: fixed`, WebKit non propaga il suo sfondo alla tela del documento, che resta trasparente e viene riempita dal sistema. Inoltre lo sfondo della radice è **l'unico** dipinto su tutta la superficie: qualunque altro elemento, anche `fixed`, viene ritagliato dal viewport e non può coprire la zona della barra home.
- **Niente altezze in vh o dvh**. In standalone risultano più basse dello schermo e lasciano scoperta una striscia in fondo. Meglio ancorare con `position: fixed` e `inset: 0`.

Le zone sicure si rispettano con `env(safe-area-inset-top)` e `env(safe-area-inset-bottom)`, così i comandi non finiscono sotto l'orologio o sotto la barra home — dove, fra l'altro, intercetterebbero il gesto di sistema.

Texture e immagini di fondo

Una texture dimensionata con `cover` viene riscalata in base all'altezza della scatola che la contiene: su elementi di altezza diversa la grana cambia scala e le giunzioni si vedono. Una **piastrella ripetuta a dimensione fissa in pixel** ha la stessa grana dappertutto per costruzione. Se l'immagine non è affiancabile, la si rende tale specchiandola nei quattro quadranti: i bordi combaciano da sé.

9. Rifarla da zero: la sequenza

1. **Supabase**: crea il progetto. Annota l'indirizzo e la chiave *anon*.
2. **SQL Editor**: crea le tabelle, attiva RLS e scrivi le policy. Senza policy l'app non vedrà niente, e senza RLS attiva vedranno tutti.
3. **Authentication**: se l'app è per poche persone, **disattiva la registrazione libera** e crea gli account a mano. Altrimenti chiunque trovi l'indirizzo può iscriversi.

4. **In locale:** scrivi `index.html`, `app.js`, `supabase.js`. Provalo con un server statico qualsiasi, per esempio `python -m http.server 8000`. Aprire il file con un doppio clic non basta: alcune funzioni del browser richiedono il protocollo HTTP.
5. **Verifica la protezione:** chiama un endpoint senza token e controlla di ricevere `[]`.
6. **GitHub:** crea il repository e caricaci i file, cartelle comprese.
7. **Vercel:** importa il repository, preset «Other», nessuna build, Deploy.
8. **iPhone:** apri l'indirizzo in Safari e aggiungi alla schermata Home.
9. **Backup:** prevedi da subito un modo per portare via i dati. Sono l'unica cosa che non sta nel repository.

10. Quando questa architettura non va bene

È adatta ad applicazioni gestite da poche persone fidate, con dati semplici e nessun segreto. Va scartata quando:

- **serve un segreto** — la chiave di un servizio a pagamento, l'invio di email, un incasso: qualunque cosa non possa stare in chiaro nel browser richiede un pezzo di codice lato server (una funzione serverless su Vercel o una Edge Function su Supabase);
- **serve logica di cui non ti puoi fidare del client:** qui la validazione è un'aggiunta cortese, e l'unica regola davvero applicata è quella scritta nelle RLS e nei vincoli della tabella;
- **servono query complesse:** PostgREST copre il caso semplice; per aggregazioni e join si passa a viste o a funzioni SQL richiamabili;
- **serve caricare file:** le immagini vanno su Supabase Storage, che è un altro servizio con altre regole di accesso;
- **serve scrivere offline:** senza rete si può leggere da una copia locale, ma la scrittura condivisa richiede una strategia di riconciliazione che questa struttura non offre da sola.

Il rischio da tenere presente: le RLS sono l'unico strato di difesa. Una policy scritta male non causa un errore visibile — apre semplicemente i dati a tutti, in silenzio. Vanno riviste ogni volta che si aggiunge una tabella.

11. Costi e alternative

Tutti e tre i servizi hanno un piano gratuito che basta per un'app familiare: repository privati illimitati su GitHub, hosting per progetti personali su Vercel, un paio di progetti su Supabase. I limiti precisi cambiano nel tempo — **vanno verificati al momento** — ma la voce a cui fare attenzione è che sui piani gratuiti i progetti Supabase **inattivi vengono messi in pausa**: se l'app viene usata di rado, la prima apertura dopo un lungo silenzio può richiedere qualche secondo.

I pezzi sono sostituibili senza cambiare l'impianto: Netlify o Cloudflare Pages al posto di Vercel; Firebase, Appwrite o PocketBase al posto di Supabase. Quello che conta è lo schema: **file statici serviti da una CDN, più un database che espone da sé un'API e applica le regole di accesso al proprio interno**. Chi ha già visto questo schema lo riconosce sotto nomi diversi.

12. In sintesi

- Nessun backend da scrivere: il database espone la propria API.
- La chiave nel client è pubblica; la sicurezza è tutta nelle policy RLS.
- Un accesso non autorizzato risponde `200` con lista vuota, non con un errore: verifica il corpo, non il codice di stato.
- Rinnova il token prima di usarlo; ritenta solo sul `401`.
- Il deploy è un commit. Non si copiano file su un server.
- Su iOS il manifest si legge all'installazione: per cambiarlo, reinstalla l'icona.
- Migrazioni numerate e idempotenti, versionate accanto al codice.
- I dati sono l'unica cosa che non sta nel repository: prevedi un export.